Session – 14

Task-1.

Create a table called Orders with columns: order_id, user_id, order_date, and total_amount. Insert at least 7 sample rows representing different users and dates, similar to how food orders appear in Zomato or Swiggy.

Solution:

```
CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  user_id INT,
  order_date DATE,
  total_amount DECIMAL(10,2)
);


INSERT INTO Orders (order_id, user_id, order_date, total_amount)
VALUES
(1, 101, '2026-10-01', 250),
(2, 101, '2026-10-03', 400),
(3, 101, '2026-10-06', 300),
(4, 102, '2026-10-02', 500),
(5, 102, '2026-10-05', 350),
(6, 103, '2026-10-01', 200),
(7, 103, '2026-10-07', 450),
(8, 101, '2026-10-10', 600);
```

Task-2.

Write a SQL query using the LAG() function to show each user's order_id, order_date, and the total_amount of their previous order (if any), ordered by user and date.

Hint: Use PARTITION BY user_id and ORDER BY order_date in your window function.

Solution:

```
SELECT
  user_id,
  order_id,
```

```
  order_date,
  total_amount,
  LAG(total_amount) OVER (
    PARTITION BY user_id
    ORDER BY order_date
  ) AS previous_order_amount
FROM Orders
ORDER BY user_id, order_date;
```

Task-3.

Using the same Orders table, write a SQL query with the LEAD() function to display each order_id, order_date, and the next order's total_amount for the same user.

Solution:

```
SELECT
  user_id,
  order_id,
  order_date,
  total_amount,
  LEAD(total_amount) OVER (
    PARTITION BY user_id
    ORDER BY order_date
  ) AS next_order_amount
FROM Orders
ORDER BY user_id, order_date;
```

Task-4.

Write a SQL query to calculate the running total of total_amount for each user, showing order_id, order_date, total_amount, and a column running_total that accumulates the sum as you move through each user's orders.

Hint: Use SUM(total_amount) OVER (PARTITION BY user_id ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Solution:

```
SELECT
  user_id,
  order_id,
  order_date,
  total_amount,
  SUM(total_amount) OVER (
    PARTITION BY user_id
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total
FROM Orders
ORDER BY user_id, order_date;
```

Task-5.

Write a SQL query to calculate a 3-order moving average of total_amount for each user, showing order_id, order_date, total_amount, and moving_avg columns.

Constraint: Use SUM() OVER() with ROWS BETWEEN 2 PRECEDING AND CURRENT ROW to compute the moving average.

Solution:

```
SELECT
  user_id,
  order_id,
  order_date,
  total_amount,
  SUM(total_amount) OVER (
    PARTITION BY user_id
    ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  )
  /
  COUNT(total_amount) OVER (
```

```
    PARTITION BY user_id
    ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_avg
FROM Orders
ORDER BY user_id, order_date;
```